COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	8/4/26	**DATE SUBMITTED:**	8/4/26
NAME:	PAYAS, JOHN PAUL S.	**INSTRUCTOR:**	SIR, JIMLORD QUEJADO

ACTIVITY NO. 5.1 Queues

1. PROCEDURE OUTPUT

ILO A:

```cpp
#include <iostream>
#include <queue>

void display(std::queue<char> copyQ);

int main() {

    std::queue<char> myQ;

    myQ.push('J');
    myQ.push('A');
    myQ.push('P');
    myQ.push('E');
    display(myQ);

    //empty
    std::cout << "Empty: " << myQ.empty() << std::endl;

    //size
    std::cout << "Size:  " << myQ.size() << std::endl;

    // front
    std::cout << "Front: " << myQ.front() << std::endl;

    // back
    std::cout << "Back:  " << myQ.back() << std::endl;

    // pop
    std::cout << "\nPopping front element " << std::endl;
    myQ.pop();


    // display after pop
    std::cout << "After pop: ";
    display(myQ);


    return 0;
}
```

```
}

void display(std::queue<char> copyQ) {

    std::queue<char> c = copyQ;

    while (!c.empty()) {
        std::cout << " " << c.front();
        c.pop();
    }
    std::cout<<"\n";
}
```

Output:

```
 J A P E
Empty: 0
Size:  4
Front: J
Back:  E

Popping front element
After pop:  A P E
```

Observation:

This code successfully demonstrates the basic operations of the queue container, including push(), pop(), front(), back(), size(), and empty(). The elements are stored and removed in a First-In, First-Out (FIFO) order, with the first inserted element being removed first. The display() function correctly prints the queue contents without modifying the original queue by using a copy. After calling pop(), the front element is removed while the remaining elements retain their correct order. Overall, the program confirms that the standard queue library provides an efficient and simple way to manage data in FIFO order

ILO B:

main.cpp

```cpp
#include <iostream>
#include "QueueArr.h"

int main()
{
    // Create an object
    QueueArr<char> q1(6);

    // ENQUEUE

    q1.Enqueue('1');
    std::cout << " q_front: " << q1.front() << " q_back: " << q1.back() << " q_size: " << q1.Size() << std::endl;
    q1.Enqueue('2');
    std::cout << " q_front: " << q1.front() << " q_back: " << q1.back() << " q_size: " << q1.Size() << std::endl;
    q1.Enqueue('3');
    std::cout << " q_front: " << q1.front() << " q_back: " << q1.back() << " q_size: " << q1.Size() << std::endl;
    q1.Enqueue('4');
    std::cout << " q_front: " << q1.front() << " q_back: " << q1.back() << " q_size: " << q1.Size() << std::endl;
    q1.Enqueue('5');
    std::cout << " q_front: " << q1.front() << " q_back: " << q1.back() << " q_size: " << q1.Size() << std::endl;
    q1.Enqueue('6');
    std::cout << " q_front: " << q1.front() << " q_back: " << q1.back() << " q_size: " << q1.Size() << std::endl;


    // Display queue
    q1.displayAll();

    // DEQUEUE

    q1.Dequeue();
    std::cout << " q_front: " << q1.front() << " q_back: " << q1.back() << " q_size: " << q1.Size() << std::endl;
    q1.displayAll();
    q1.Dequeue();
    std::cout << " q_front: " << q1.front() << " q_back: " << q1.back() << " q_size: " << q1.Size() << std::endl;
    q1.displayAll();
    q1.Dequeue();
    std::cout << " q_front: " << q1.front() << " q_back: " << q1.back() << " q_size: " << q1.Size() << std::endl;
    q1.displayAll();
    q1.Dequeue();
    std::cout << " q_front: " << q1.front() << " q_back: " << q1.back() << " q_size: " << q1.Size() << std::endl;
    q1.displayAll();
    q1.clear();
    std::cout << " q_front: " << q1.front() << " q_back: " << q1.back() << " q_size: " << q1.Size() << std::endl;
    q1.displayAll();
    q1.clear();
    std::cout << " q_front: " << q1.front() << " q_back: " << q1.back() << " q_size: " << q1.Size() << std::endl;
    q1.displayAll();
    q1.Dequeue();
```

```cpp
    std::cout << " q_size: " << q1.Size() << std::endl;

    q1.displayAll();


    return 0;
}
```

queueLL.h

```cpp
#ifndef QUEUEARR_H
#define QUEUEARR_H

#include <iostream>
#include <cstddef>

template <typename T>
class QueueArr
{
private:
    T* q_array;
    size_t q_capacity;
    size_t q_size;
    int q_front;
    int q_back;

public:
    QueueArr(size_t capacity);
    QueueArr();

    ~QueueArr();

    bool Empty() const;
    bool Full() const;

    size_t Size() const;
    void clear();

    T front() const;
    T back() const;

    void Enqueue(T newData);
    void Dequeue();

    void displayAll() const;

    QueueArr(const QueueArr<T>& other);
    QueueArr<T>& operator=(const QueueArr<T>& other);
};


// Constructor
template <typename T>
QueueArr<T>::QueueArr(size_t capacity)
{
    q_capacity = capacity;
    q_array = new T[q_capacity];
    q_size = 0;
```

```
}


// Default Constructor
template <typename T>
QueueArr<T>::QueueArr()
{
    q_capacity = 10;
    q_array = new T[q_capacity];
    q_size = 0;
    q_front = 0;
    q_back = -1;
}


// Destructor
template <typename T>
QueueArr<T>::~QueueArr()
{
    delete[] q_array;
}


// Empty
template <typename T>
bool QueueArr<T>::Empty() const
{
    return q_size == 0;
}


// Full
template <typename T>
bool QueueArr<T>::Full() const
{
    return q_size == q_capacity;
}


// Size
template <typename T>
size_t QueueArr<T>::Size() const
{
    return q_size;
}
```

```cpp
// Front
template <typename T>
T QueueArr<T>::front() const
{
    if (Empty())
    {
        std::cout << " Queue is empty" << std::endl;
        return T();
    }
    return q_array[q_front];
}


// Back
template <typename T>
T QueueArr<T>::back() const
{
    if (Empty())
    {
        std::cout << " Queue is empty" << std::endl;
        return T();
    }
    return q_array[q_back];
}


// Enqueue
template <typename T>
void QueueArr<T>::Enqueue(T newData)
{
    if (Full())
    {
        std::cout << " Queue is full" << std::endl;
        return;
    }
    q_back = (q_back + 1) % q_capacity;
    q_array[q_back] = newData;
    q_size++;
}
```

```cpp
// Dequeue
template <typename T>
void QueueArr<T>::Dequeue()
{
    if (Empty())
    {
        std::cout << " Queue is empty" << std::endl;
        return;
    }
    q_front = (q_front + 1) % q_capacity;
    q_size--;

    if (q_size == 0)
    {
        clear();
    }
}


// Clear
template <typename T>
void QueueArr<T>::clear()
{
    q_size = 0;
    q_front = 0;
    q_back = -1;
}


// Copy Constructor
template <typename T>
QueueArr<T>::QueueArr(const QueueArr<T>& other)
{
    q_capacity = other.q_capacity;
    q_size = other.q_size;
    q_front = other.q_front;
    q_back = other.q_back;

    q_array = new T[q_capacity];

    for (size_t i = 0; i < q_capacity; i++)
    {
        q_array[i] = other.q_array[i];
    }
}
```

```cpp
// Assignment Operator
template <typename T>
QueueArr<T>& QueueArr<T>::operator=(const QueueArr<T>& other)
{
    if (this != &other)
    {
        delete[] q_array;

        q_capacity = other.q_capacity;
        q_size = other.q_size;
        q_front = other.q_front;
        q_back = other.q_back;

        q_array = new T[q_capacity];

        for (size_t i = 0; i < q_capacity; i++)
        {
            q_array[i] = other.q_array[i];
        }
    }
    return
     *this;
}


//Display all
template <typename T>
void QueueArr<T>::displayAll() const
{
    if (Empty())
    {
        std::cout << " The queue is empty!" << std::endl;
        return;
    }
    std::cout << " The current Queue: ";

    for (size_t i = 0; i < q_size; i++)
    {
        int index = (q_front + i) % q_capacity;
        std::cout << q_array[index] << " ";
    }
    std::cout << std::endl;
}


#endif
```

Output:

```
q_front: 1 q_back: 1 q_size: 1
q_front: 1 q_back: 2 q_size: 2
q_front: 1 q_back: 3 q_size: 3
q_front: 1 q_back: 4 q_size: 4
q_front: 1 q_back: 5 q_size: 5
q_front: 1 q_back: 6 q_size: 6
The current Queue: 1 2 3 4 5 6
q_front: 2 q_back: 6 q_size: 5
The current Queue: 2 3 4 5 6
q_front: 3 q_back: 6 q_size: 4
The current Queue: 3 4 5 6
q_front: 4 q_back: 6 q_size: 3
The current Queue: 4 5 6
q_front: 5 q_back: 6 q_size: 2
The current Queue: 5 6
q_front:  Queue is empty
q_back:  Queue is empty
q_size: 0
The queue is empty!
q_front:  Queue is empty
q_back:  Queue is empty
q_size: 0
The queue is empty!
Queue is empty
q_size: 0
The queue is empty!
```

Observation:

The program demonstrates the implementation of a queue using a linked list, where elements are added at the rear using enqueue() and removed from the front using dequeue(). The displayAll() function correctly displays the contents of the queue, while getfront() and getback() identify the first and last elements in the queue. The output shows that the queue follows the First-In, First-Out (FIFO) principle, as the earliest inserted element is always removed first. After all elements are dequeued, the program correctly indicates that the queue is empty before allowing new elements to be inserted again. Overall, the linked-list implementation provides a dynamic queue that can efficiently perform insertion, deletion, and retrieval operations without requiring a fixed-size array.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

COMPUTER ENGINEERING

QUEZON CITY

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Chat gpt**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **It just fixed the grammar, but it was my idea**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.